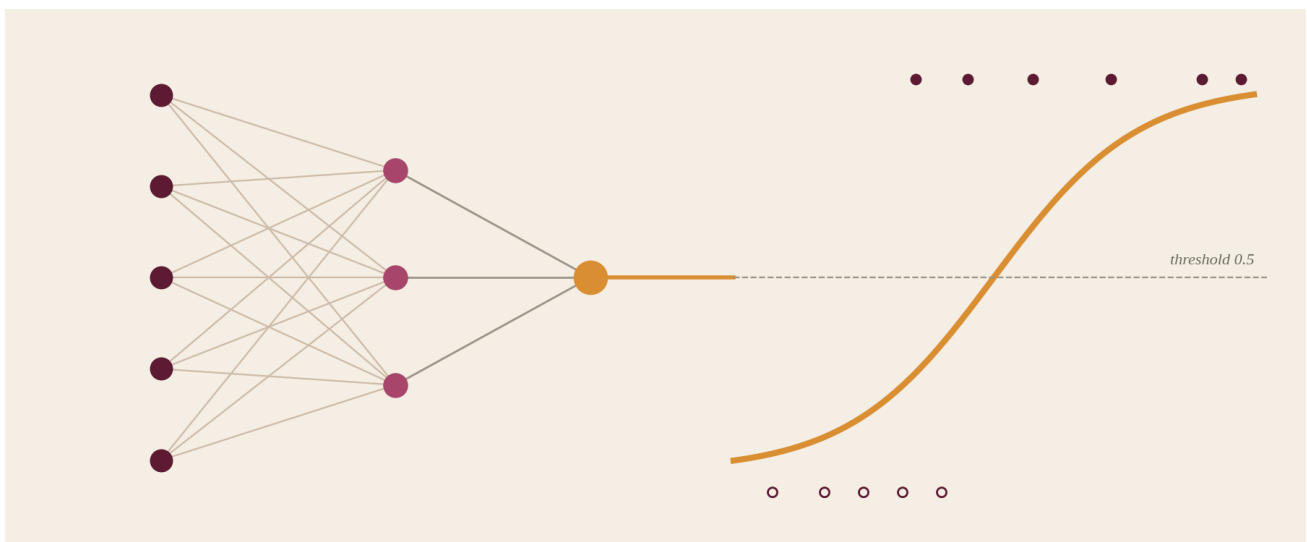


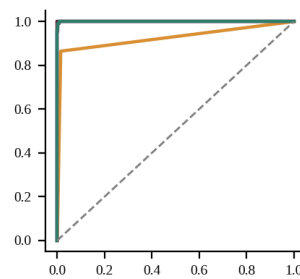
RISK ALERT CLASSIFIER

Part H: Final Analysis & Reporting

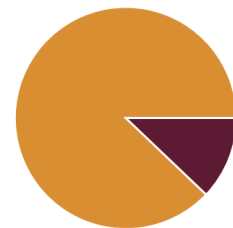


Safe	809	0
Risky	0	111
	Safe	Risky

Confusion Matrix



ROC - AUC



Class balance

Charts: confusion matrix and ROC curves of the four models on the 920-customer test set, and the class balance of the data.

Part H: Final Analysis & Reporting

Final report for the Risk Alert Classifier notebook (Q26). All results are from the same 920-customer test set.

4,600 customers	12.11% risky (557 customers)	3,680 / 920 train / test rows	111 risky customers in test set
---------------------------	--	---	---

Missing values appear in **7 of the 19 columns**, none above 5% of the 4,600 rows (credit_score is highest with 216). Numeric columns were filled with KNN imputation.

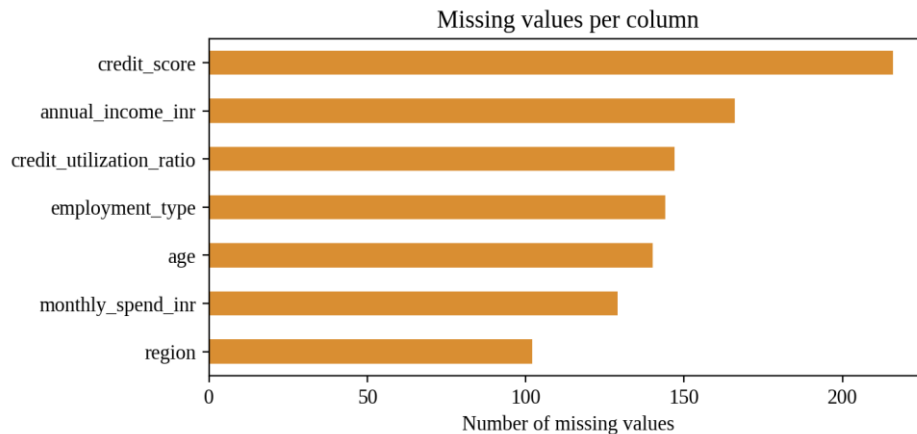


Figure 1. Missing values per column.

1 Best classification model and justification

The final model is **Logistic Regression**. It scored **1.0** on accuracy, precision, recall, F1 and AUC-ROC. Its confusion matrix is $[[809, 0], [0, 111]]$: every risky customer was caught and no safe customer was flagged.

- **Matches the business goal.** The goal is to minimise false negatives (Q25), so recall decides. Logistic Regression has recall 1.0, against 0.9910 for Random Forest and Tuned Random Forest (1 missed each) and 0.8649 for the Decision Tree (15 missed).
- **Best on every other metric too:** 0 false positives and AUC-ROC of 1.0.
- **Simple and explainable.** It is fast and easy to interpret, which suits a risk team.

Caveat before deployment

The test set holds only 111 risky customers, so the gap between 0 and 1 missed customer is small. Tuned Random Forest (accuracy 0.9978, 1 false negative, 1 false positive) is a close second.

customer_id is currently used as a feature and should be removed. Confirm the result with cross-validation.

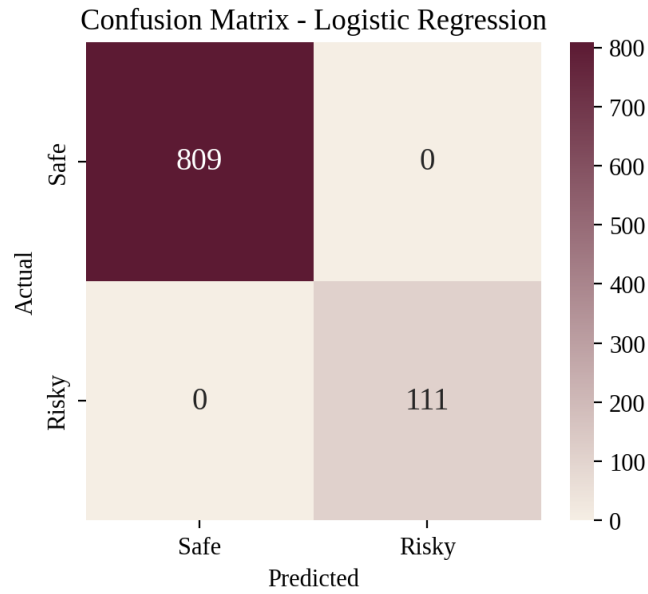


Figure 2. Confusion matrix of the final model: 809 safe and 111 risky customers all classified correctly.

2 Impact of imbalance handling techniques

Only **557 of 4,600 customers (12.11%)** are risky, about 7 safe customers for every risky one. A model that always said "safe" would be about 88% accurate and catch nothing. Four techniques were applied to the training data only and compared with the baseline Logistic Regression.

Class distribution of risk_status

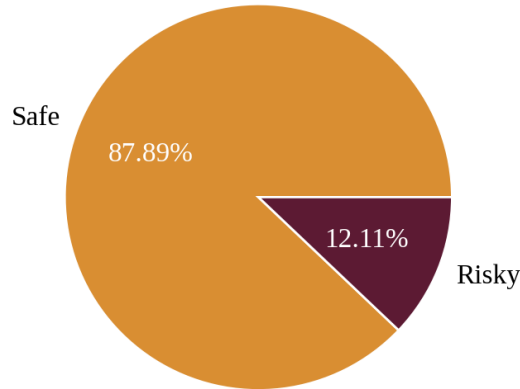


Figure 3. Class distribution of risk_status: 87.89% safe and 12.11% risky.

Setting	Recall	F1-Score	AUC-ROC	False positives
Before balancing	1.0000	1.0000	1.0000	0
Under-Sampling	1.0000	0.9823	1.0000	4
Over-Sampling	1.0000	0.9867	1.0000	3
SMOTE	1.0000	0.9911	1.0000	2
ADASYN	1.0000	0.9694	0.999978	7

Table 1. Logistic Regression before and after balancing (test set).

- **Recall stayed at 1.0** for every technique, so the baseline had nothing to improve.

- **F1 fell** because precision fell. Balancing added false alarms: 4 (under-sampling), 3 (over-sampling), 2 (SMOTE) and 7 (ADASYN), against 0 before.
- **SMOTE** was the least harmful and **ADASYN** the most harmful. ADASYN was also the only method with AUC below 1.0.

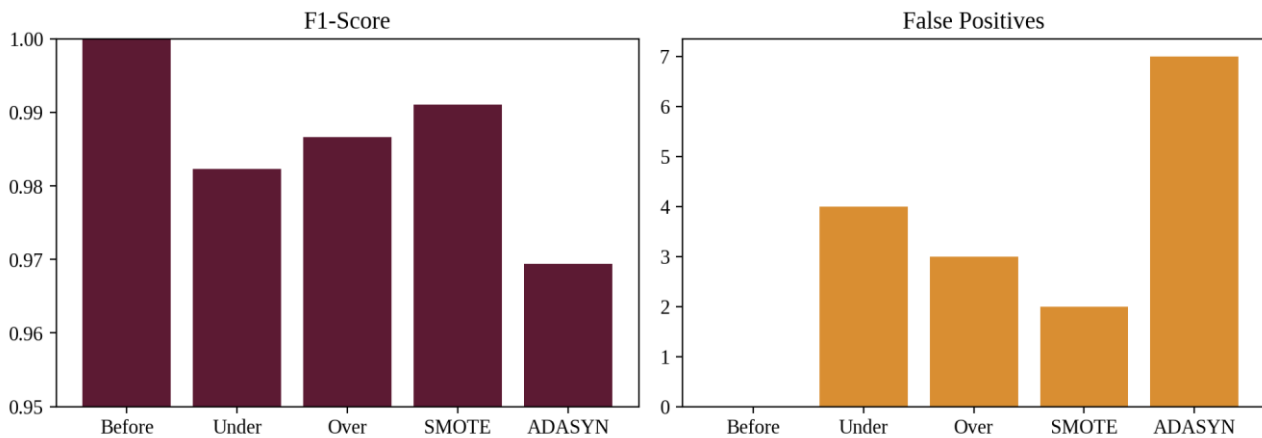


Figure 4. F1-Score and false positives of Logistic Regression before and after each balancing technique.

Takeaway

Imbalance handling was not needed for this dataset. It did not raise recall and only added false positives.

3 Comparison of performance metrics

The four models were evaluated on the same test set. **Logistic Regression** is perfect on every metric; the tree-based models trail slightly.

Model	Accuracy	Precision	Recall	F1-Score	AUC-ROC
Logistic Regression	1.0000	1.0000	1.0000	1.0000	1.0000
Decision Tree	0.9707	0.8889	0.8649	0.8767	0.9250
Random Forest	0.9967	0.9821	0.9910	0.9865	0.9999
Tuned Random Forest	0.9978	0.9910	0.9910	0.9910	0.9999

Table 2. Test-set metrics. Precision, F1 and counts for some models were recomputed by re-running the notebook code; they reproduce every figure it printed.

- **Decision Tree overfits:** training accuracy 1.0 against test accuracy 0.9707, with the lowest AUC-ROC (0.9250) and recall (0.8649).
- **Random Forest generalises better:** test accuracy 0.9967 (3 errors in 920 rows).
- **Tuning helped only slightly:** accuracy rose from 0.9967 to 0.9978 and F1 from 0.9865 to 0.9910, which is 3 errors down to 2.

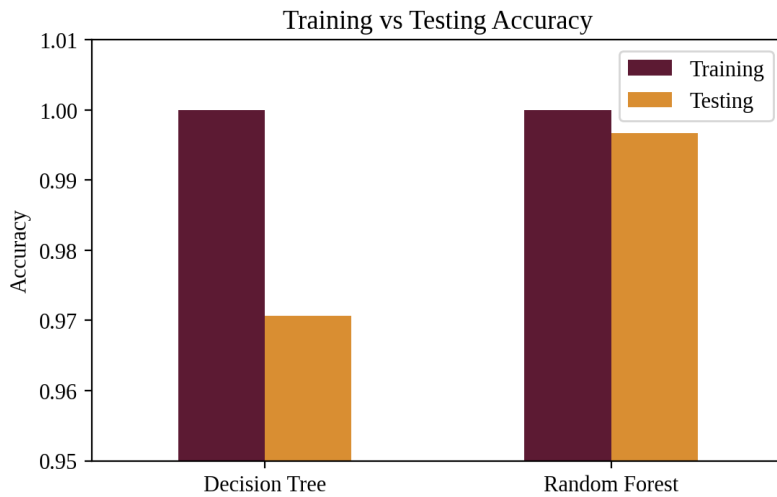


Figure 5. Training vs testing accuracy (y-axis starts at 0.95).

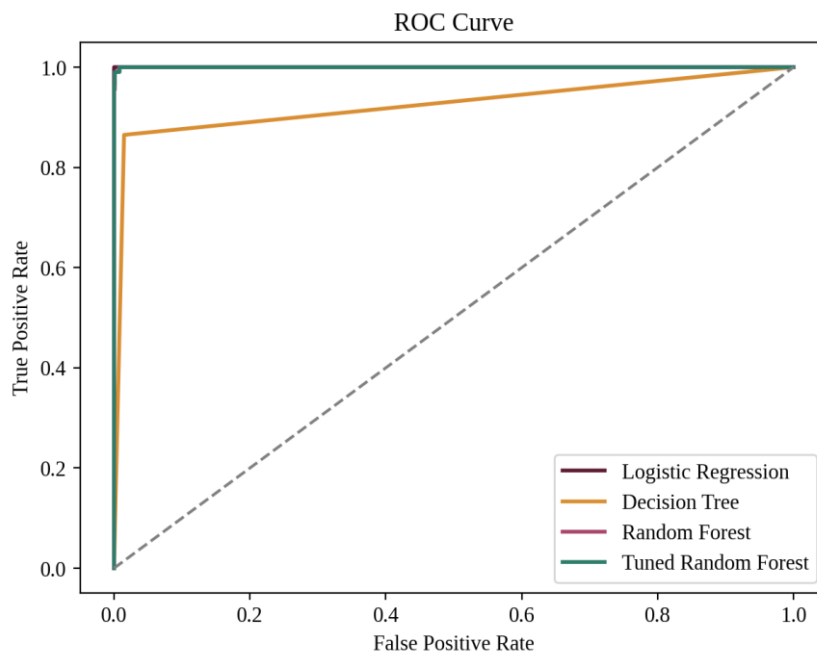


Figure 6. ROC curves. The three near-perfect curves overlap; the Decision Tree curve is clearly lower.

4 Business interpretation of false positives and false negatives

Error	Meaning	Business effect	Final model
False positive (Type-I)	A safe customer is flagged as risky	False alarm: wasted review effort and a possible lost good customer	0 of 809 safe customers
False negative (Type-II)	A risky customer is passed as safe	Missed risk, for example a loan approved that later defaults. Usually the costlier error	0 of 111 risky customers

Because false negatives are costlier, models were ranked by recall. Across all eight configurations, only the tree-based models missed risky customers: **Decision Tree 15**, **Random Forest 1** and **Tuned Random Forest 1**. False alarms ranged from 0 (Logistic Regression) to 12 (Decision Tree).

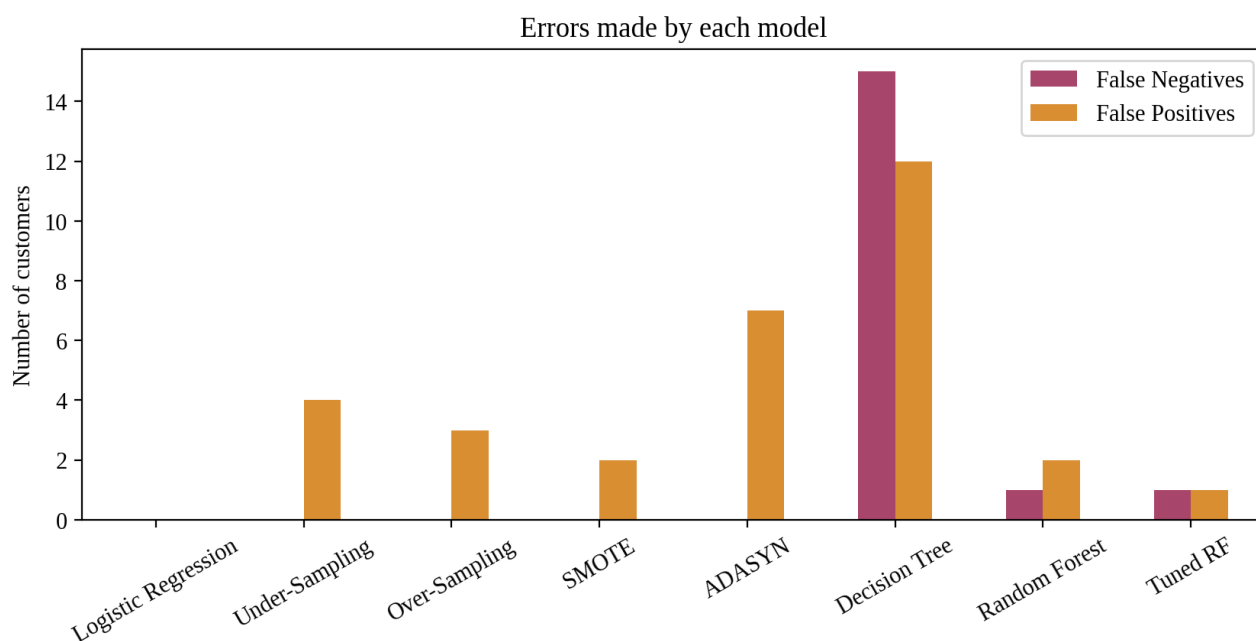


Figure 7. False negatives and false positives for every model configuration (test set).

The dataset has no cost figures, so no money values are given. A cost-weighted comparison would need the loss per default and the cost of a manual review.

Conclusion and next steps

- 1. Deploy Logistic Regression** as the risk alert model: zero false negatives and zero false positives on the test set.
- 2. Skip resampling** for this dataset; it only added false alarms.
- 3. Clean the features:** remove customer_id. Gender, region, employment type and last transaction date are not used by the models at present.
- 4. Validate** with cross-validation and monitor false negatives after deployment.